

## Lecture 6

### Lemmatization

Lemmatization means finding the original base word (dictionary form) of a word, even if it looks different.

#### Example:

- am, are, is → **be**
- car, cars, car's → **car**

#### Real life example:

When searching online, if you search “**running**”, the system understands it as “**run**” so it can show better results.

It helps computers know that different forms of a word have the same meaning.

---

### Stemming

Stemming is a simpler method than lemmatization.

It just removes the ending part of a word without checking the correct dictionary form.

#### Example:

- playing → **play**
- studies → **studi**

(“studi” is not a real word, so stemming is less accurate)

#### Real life example:

Search engines use stemming to quickly match words like **connected**, **connecting**, and **connection**.

---

### Sentence Segmentation

Sentence segmentation means dividing a paragraph into separate sentences.

#### Example:

Paragraph:

Shelby is a student. She is sick today. She will not go to school.

After segmentation:

- Shelby is a student
- She is sick today
- She will not go to school

**Real life example:**

Mobile phones and chat apps use sentence segmentation to understand where one sentence ends and the next starts.

**Note:**

- ? and ! are easy sentence endings
- . is sometimes confusing because of:
  - Dr. Ahmed
  - Inc.
  - 4.3
  - 0.02%

## Lecture 8

### 1. Word-based Tokenization

It means splitting text into words using spaces and punctuation.

**Example:**

“I love cats” → I | love | cats

**Real life example:**

Google search splits your sentence into separate words to understand your search.

**Advantages:**

- Very easy to use
- Easy to understand

**Disadvantages:**

- “Let” and “Let’s” become different words
- Wrong spelling becomes a new token
- Chinese language has no spaces
- Vocabulary becomes very large

---

## 2. Character-based Tokenization

It means splitting text into single characters.

### Example:

“Cat” → C | a | t

### Real life example:

Used in password checking or spelling correction systems.

### Advantages:

- Almost no unknown words
- Good for languages where characters have meaning
- Easy to apply

### Disadvantages:

- One character usually has no full meaning
- Hard to understand full word meaning
- More input for computer to process

---

## 3. Subword Tokenization

It splits rare words into smaller meaningful parts but keeps common words complete.

### Example:

“Tokenization” → Token | ##ization

### Real life example:

Used in AI tools like chatbots and translators.

### Advantages:

- Solves unknown word problem
- Smaller vocabulary size

### Disadvantages:

- More complex method

### Examples:

**BPE** and **WordPiece**

---

## Byte Pair Encoding (BPE)

BPE is a subword tokenization method.

It starts from single characters and keeps joining the most common pair.

---

### Training Example

Words:

- low
- lowest
- newer
- wider
- new

#### First Vocabulary:

`_ , d , e , i , l , n , o , r , s , t , w`

(`_` means end of word)

---

#### Most Frequent Pair:

`e + r = er`

Because “er” appears 9 times.

So new vocabulary becomes:

`_ , d , e , i , l , n , o , r , s , t , w , er`

This is the final vocabulary after this merge.

---

## Using BPE

### Example 1:

Input: **newer**

Tokens:

**newer**

because full merge exists

---

### **Example 2:**

Input: **lower**

Tokens:

**low | er**

because “low” and “er” are learned tokens

---

### **Stopwords Removal**

Stopwords are very common words like:

**the, of, and, to**

These words usually do not add important meaning.

#### **Example:**

“I am going to the school”

After removing stopwords:

**going school**

#### **Real life example:**

Search engines remove stopwords to save space and improve speed.

#### **Problem:**

Sometimes stopwords are important

Example:

- “to be or not to be”
- “flights to Portland”

So we must be careful.

---

### **Token / Term Normalization**

It means making different forms of a word look the same.

#### **Example:**

- U.S.A → USA
- color → colour

- anti-discriminatory → antidiscriminatory

**Real life example:**

Google understands “color” and “colour” as same meaning.

---

**Stemming**

Stemming means removing word endings to get the root form.

**Example:**

- fish
- fishes
- fishing

→ **fish**

**Real life example:**

If you search “fishing”, the system can also show results for “fish”.

**Types:**

- Aggressive → fish and fishing same
- Conservative → only remove simple plural “s”
- No stemming → keep all words separate

**Problem:**

Sometimes wrong results happen

Example:

**Centuries → Centurie**

(not a correct word)

## Lecture 9

**Stemming**

Stemming means reducing words to a common root (stem).

**Example:**

- fish
- fishes
- fishing

→ **fish**

It helps group similar words together.

**Real life example:**

If you search “**fishing**”, the search engine may also show results for “**fish**”.

---

## **Types of Stemming**

### **1. Aggressive Stemming**

It removes more letters.

Example:

fish and fishing → **fish**

### **2. Conservative Stemming**

It removes only small endings like plural **s**

Example:

cats → **cat**

### **3. No Stemming**

All words stay separate

Example:

fish, fishes, fishing all remain different

---

## **Problems in Stemming**

### **Over-stemming**

Too much part of the word is removed.

Example:

- centennial
- century
- center

→ all become **cent** (wrong)

---

### **Under-stemming**

Too little part is removed.

Example:

- acquire
- acquired → acquir
- acquisition → acquis

These should be grouped but are different.

---

### **Porter Stemmer**

It is the most common English stemmer made by **Martin Porter**.

It removes endings like:

- ing
- sses
- eed

Example:

agreed → agree

#### **Advantage:**

- Better results than many other stemmers
- Less error rate

#### **Disadvantage:**

- Sometimes gives stems that are not real words
- 

### **Krovetz Stemmer**

Made by **Robert Krovetz**

It first checks the dictionary.

If word exists → keep it

If not → remove suffix and check again

#### **Advantage:**

- Produces real words, not broken stems

#### **Disadvantage:**

- Can miss some correct matches



---

## **FP, FN, TP, TN**

These are used to check system accuracy.

---

### **True Positive (TP)**

System says correct, and it is correct.

#### **Example:**

Spam email is detected as spam.

---

### **True Negative (TN)**

System says not correct, and it is really not correct.

#### **Example:**

Normal email is detected as normal.

---

### **False Positive (FP)**

System says correct, but it is wrong.

#### **Example:**

Policy → Police (wrong match)

A normal email marked as spam.

---

### **False Negative (FN)**

System says wrong, but it should be correct.

#### **Example:**

noise and noisy are not matched

Spam email marked as normal.

---

## **Lemmatization**

Lemmatization means finding the correct dictionary base form of a word.

#### **Example:**

- am, are, is → **be**
- car, cars, car's → **car**

#### **Sentence Example:**

“The boy’s cars are different colors”

→ **the boy car be different color**

#### **Real life example:**

When searching “**running**”, the system understands it as “**run**” for better results.

#### **Difference from Stemming:**

- Stemming cuts letters only
- Lemmatization finds the correct real word

### **Phrases**

A phrase is a group of words used together.

#### **Example:**

- red apple
- black sea
- University of Southern Maine

Phrases are more clear than single words.

Example:

**apple** is general

**red apple** is more specific

#### **Real life example:**

Google search gives better results for “**red shoes**” than only “**shoes**”

### **N-gram**

N-gram means a group of words together.

#### **Types:**

##### **Unigram**

1 word

Example:  
apple

### **Bigram**

2 words

Example:  
red apple

### **Trigram**

3 words

Example:  
I love cats

The more a phrase repeats, the more meaningful it is likely to be.

## **Lecture 10**

### **Text Classification (Text Categorization)**

Text classification means giving a label or category to a full text or document.

#### **Example:**

Email → Spam or Not Spam  
Movie review → Positive or Negative

---

### **Common Text Classification Tasks**

---

#### **1. Sentiment Analysis**

It finds whether the text feeling is **positive** or **negative**.

#### **Example:**

“This movie is amazing!” → **Positive**  
“This pizza is awful” → **Negative**

#### **Real life example:**

Online shopping websites check customer reviews to know if people like a product.

---

#### **2. Spam Detection**

It checks whether an email is **spam** or **normal**.

**Example:**

“You won 1 million dollars!” → **Spam**

**Real life example:**

Gmail automatically puts fake promotion emails into spam folder.

---

### **3. Authorship Identification**

It finds who wrote the text.

**Example:**

Finding whether a poem was written by a specific writer.

**Real life example:**

Used in crime investigation for unknown messages.

---

### **4. Age / Gender Identification**

It guesses writer details like age or gender.

**Example:**

Text style may show if writer is young or old.

**Real life example:**

Used in marketing to understand customers.

---

### **5. Language Identification**

It finds which language is used.

**Example:**

“Bonjour” → French

**Real life example:**

Google Translate first detects the language automatically.

---

### **Simple Sentiment Analysis**

This is usually **Binary Classification**

Only two classes:

- Positive (+)
- Negative (-)

**Example:**

“Awesome food, I love this place!” → **Positive**

“Awful pizza and overpriced” → **Negative**

---

**Why Sentiment Analysis?**

It helps in many areas:

**Movies**

Is review positive or negative?

**Products**

Do people like the new iPhone?

**Politics**

What do people think about a candidate?

**Prediction**

Can public opinion help predict elections or market trends?

---

**Text Classification Formula**

**Input:**

- A document (text)
- A fixed set of classes

Example:

Classes = {Spam, Not Spam}

**Output:**

- One predicted class

Example:

Email → Spam

---

## Classification Methods

---

### 1. Hand-coded Rules

Humans write rules manually.

#### Example:

If email contains:

- “dollars”
- “you have been selected”

→ classify as **Spam**

#### Advantage:

- Can be accurate

#### Disadvantage:

- Expensive to make
- Hard to maintain
- Rules can break easily

#### Real life example:

Old email filters used rule-based spam detection.

---

### 2. Supervised Machine Learning

The computer learns from already labeled examples.

#### Example:

Many movie reviews already marked as positive or negative

The model learns from them and predicts for new reviews.

#### Real life example:

Netflix review systems use this idea.

---

### Some Classifiers

- Naïve Bayes
- Logistic Regression

- Support Vector Machine (SVM)
  - k-Nearest Neighbors (KNN)
- 

### **Naïve Bayes Classifier**

It is based on **Bayes Rule**.

It assumes features are simple and independent.

It uses probability to decide the class.

---

### **Bag of Words**

A document is treated like a bag of words.

Word order is ignored.

Only word frequency matters.

#### **Example:**

“I love pizza”

“Pizza love I”

Both are treated almost the same.

#### **Real life example:**

Spam detection often uses bag of words.

---

### **Problem with MLE**

If a word never appeared in training data, probability becomes **zero**.

#### **Example:**

Word = fantastic

If it never appeared in positive reviews, system may fail.

Zero probability creates problems.

---

### **Solution: Add-One (Laplace) Smoothing**

We add 1 to every word count.

This avoids zero probability.

**Example:**

Even if “fantastic” never appeared, it still gets a small probability.

---

**Unknown Words Problem**

Sometimes test data has a word not in vocabulary.

**Example:**

Training data has no word:

“superb”

Test review contains:

“This phone is superb”

**Simple solution:**

Remove that unknown word from test document.